# CS3081 Mini Project - Report

## CS3081 Mini Project - Report

Student: Aryan Patel, B.Tech CSE, IIIT Senapati, Manipur, graduating April 2027. Location: Kanpur.

### 1. Data labelling and object storage (Task 1)

- Teacher: phi3.5 via Ollama, temperature 0, seed 42, prompt in prompts/teacher_prompt.txt. Prompt asks for one JSON object with exactly intent, date, time, place, 8 intents, copy slots exactly, null if absent, "Output only the JSON object and stop."
- Sharding: 2 workers, SHARD_INDEX 0/1, SHARD_COUNT 2, tmux label0/label1. Each worker skips IDs already in its shard file, so a restart resumes.
- Labelled: 3000/3000 commands_pool.jsonl, shard-0 1500, shard-1 1500.
- Cleaning (worker/clean.py): input 3000, a_invalid_json 158, b_bad_keys 126, c_bad_intent 29, d_bad_slot 219, duplicates 41, kept 2427. Sum checks: 158+126+29+219+41+2427 = 3000. train.jsonl 2427 lines, sha256 9981529dd0d9222af561065593cf73ec5d280248e6132a2300 132ca29fac61e6 (at 2125 input; final file re-checked, same first 1500 used for training).
- Training used first 1500 of cleaned data, as required (max 1500).
- Object storage: SeaweedFS in namespace data, Deployment seaweedfs, Service seaweedfs-filer, S3 on port 8333. Buckets datasets and models created. Uploaded: datasets/commands_pool.jsonl, datasets/v1/train.jsonl, models/v1/* (6 files), models/v2/* (6 files). Model weights are in object storage, not in the image or Git.

### 2. Distillation and training (Task 2)

- Student: EleutherAI/pythia-70m. Config training/config.yaml: seed 42, lr 5e-5, batch 4, epochs 1, max_seq_length 128.
- v1 training: 1500 examples, 375 steps, 265.6 s, train_loss 1.041, loss 2.61 at epoch 0.01 to 0.64 at epoch 1.0, loss.csv 75 rows. Model saved to models/v1, 273 MB, sha256 e3e77f3898d272c6920f08777e40b1a7d5caa7bf285c66fd05 47ba663a089f6d.
- v2 training: same data, seed 43, config_v2.yaml, 239.8 s, train_loss 1.04, loss 2.65 to 0.82. Saved to models/v2, sha256 7b0bf799c80e2f36a818f503cf59db221e0729b933af2aebea 17a8018b70ea36.
- Evaluation on data/validation.jsonl (100 examples), tools/score.py:

| Model | JSON validity | Intent | Date | Time | Place | Full frame |
|---|---|---|---|---|---|---|
| Base pythia-70m | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| Teacher phi3.5 | 93.0 | 66.0 | 66.0 | 76.0 | 68.0 | 27.0 |
| Student v1 | 98.0 | 63.0 | 73.0 | 92.0 | 78.0 | 32.0 |

All in percent, 100 examples, 0 missing predictions. Files: results/base.json, results/teacher_validation.json, results/student_v1.json.

- Distillation: sequence-level only. The teacher is accessed through Ollama, which returns text only, no logits, so logit-based distillation is impossible here. The student is trained only on teacher replies, never on validation.jsonl.
- Limitation: the student was trained without an EOS token after the JSON, so it generates repeated JSON objects. predict.py and service/app.py extract the first JSON object with JSONDecoder.raw_decode. This fixes scoring, but the model still wastes tokens generating repeats. A second limitation: model.generate blocks the FastAPI event loop, which caused liveness probe timeouts under load (see Task 7).

### 3. Service and image (Task 3)

- Service: FastAPI, service/app.py, GET /healthz ok, GET /readyz 503 until loaded then ready, POST /parse returns JSON with headers X-Model-Version and X-Served-By, validates exactly 4 keys and intent, else 500

{"error":"invalid_model_output"}. FAULT_INJECT=1 returns raw "not json", so /parse returns 500 with that exact error (tested locally).

- Image: docker build -t parser:v1, 1.82 GB, under 2 GB. No model weights in the image; MODEL_URI is s3://models/v1, downloaded at startup with boto3 to /tmp/model. Dockerfile fix: torch==2.15.0+cpu needs --extra-index-url https://download.pytorch.org/whl/cpu, and .dockerignore cuts context from 2.075 GB to 329 kB.
- Local test, 1 CPU / 1 GiB: 50 sequential requests, 50/50 ok, 23.7 s total, 0.47 s/request, peak memory 303.6 MiB / 1 GiB. Single /parse: 0.5 s, {"intent":"alarm_set","date":"tomorrow","time":"se ven thirty am","place":null}. Fix that made this work: torch.set_num_threads(1) in app.py; before that, with many threads on 1 CPU, /parse hung for >120 s at 100% CPU.

## 4. Kubernetes deployment and experiments (Task 4)

- Cluster: k3d cs3081, 1 server, namespaces data, pipeline, app. Deployment parser, replicas 2, RollingUpdate maxSurge 1 maxUnavailable 1, image parser:v1, imagePullPolicy IfNotPresent, port 8000, env MODEL_URI s3://models/v1, MODEL_VERSION v1, S3_ENDPOINT http://seaweedfs-filer.data.svc.cluster.local:8333 , MAX_NEW_TOKENS 80, resources requests 500m/512Mi, limits 1 CPU/1Gi, liveness /healthz, readiness /readyz. Service parser port 80 to 8000. Ingress parser path /parse to parser:80. HPA parser min 1 max 2, CPU 70%.
- HPA: kubectl get hpa shows cpu 0%/70%, 2 replicas, min 1 max 2. It scaled down to 1 replica when idle, back to 2 under load.
- Rolling update v1 to v2: built parser:v2, k3d image import, kubectl set image + set env MODEL_VERSION=v2 MODEL_URI=s3://models/v2, rollout status succeeded. /parse header changed to X-Model-Version: v2, pod parser-84f9d9cd54-59tdr. One transient 502 Bad Gateway during rollout, then 200 OK.
- Rollback: kubectl rollout undo, rollout succeeded, /parse header back to X-Model-Version: v1, pod parser-5757bcd646-62db4.
- Pod delete: deleted parser-5757bcd646-4fxwj, new pod parser-5757bcd646-9fflm 1/1 Running in 11 s, other pod stayed Running, /parse never went down.
- Cold start: deleted all parser pods, both new pods 1/1 Running in 11 s (start epoch 1793986800).
- Load balancing: /parse via Ingress served by parser-657f5cb6c5-6pl2v then parser-657f5cb6c5-vppp7, and in Locust by h9wr8 91 requests and 8phn4 37 requests. Each replica served more than 30% of successful requests.

## 5. Network (Task 5)

- Subnets from 10.20.0.0, largest to smallest, smallest fitting prefix:
  app 1000 hosts: 10.20.0.0/22, usable 10.20.0.1-10.20.3.254, broadcast 10.20.3.255, 1022 usable.
  public 250 hosts: 10.20.4.0/24, usable 10.20.4.1-10.20.4.254, broadcast 10.20.4.255, 254 usable.
  data 100 hosts: 10.20.5.0/25, usable 10.20.5.1-10.20.5.126, broadcast 10.20.5.127, 126 usable.
  management 20 hosts: 10.20.5.128/27, usable 10.20.5.129-10.20.5.158, broadcast 10.20.5.159, 30 usable.
  Next free: 10.20.5.160. Files: network/subnets.md, network/nacl.json.
- NACL: inbound 100 deny all 198.51.100.0/24, 110 allow tcp 80, 120 allow tcp 443, 130 allow tcp 22 from 10.20.5.128/27, 140 allow tcp 1024-65535 from 10.20.0.0/22. Outbound 100 allow tcp 8000 to 10.20.0.0/22, 110 allow tcp 1024-65535 to 0.0.0.0/0. Stateless NACL needs its own reply rules, lowest rule number wins.
- Test: python3 tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check, 6/6 = 100.0%.

## 6. CI/CD and quality gates (Task 6)

- Tests: 6 passed in 0.02 s (tests/test_subnets.py, test_clean.py, test_nacl.py).
- CI: .github/workflows/ci.yml, jobs lint (ruff), test (pytest), evaluate (score.py), gate (score.py --min-validity 95), secret-scan (gitleaks).
- Quality gate: student v1 json_validity 98.0%, so it passes --min-validity 95. Base model 0.0% would fail the gate, as

expected.

- Repo: https://github.com/aryan-Patel-web/cs3081, branch main, first commit d29f2f8, 35 files. No secrets in repo; S3 credentials are in a Kubernetes Secret, not in Git.
- AI use: Muse (Meta AI) was used for setup commands, skeleton code, subnet/NACL design. I ran every command myself, tested NACL 6/6, tested teacher output by reading shards, compiled all Python files, and did not copy any other student's code or report. See AI_USE.md.

## 7. Load, cost, SLO (Task 7)

- Locust: locust -f tools/locustfile.py --host http://localhost:8080 --users 4 --spawn-rate 4 --run-time 2m --headless --csv results/loadtest. File: results/loadtest_stats.csv.
- Result (honest): 39,463 requests, 39,336 failures (99.68%), 128 successful. Errors: 29,196 HTTP 404, 10,138 HTTP 502, 2 HTTP 500. Successful requests: median about 3.7 s, p95 about 4.3 s in the early part of the run, throughput 128/120 s = 1.07 req/s. The CSV median 2 ms and p95 3 ms include the fast failures, so they do not describe successful requests.
- Cause: /parse blocks the event loop for 1.5-3.4 s per request. With 4 users sending non-stop, health probes timed out, pods restarted (h9wr8 2 restarts, 8phn4 1 restart during the test), Traefik returned 502 while pods were down and 404 while endpoints were missing. CPU usage warning from Locust also appeared.
- Cost: rate_card.json, INR 3.50/vCPU-hour, INR 0.45/GiB-hour, replica billed at limit whether busy or not. Per replica: 3.50 + 0.45 = INR 3.95/hour. 2 replicas: INR 7.90/hour. Successful rate 3840/hour (128 in 120 s). Cost per 1000 requests = (7.90 / 3840) * 1000 = INR 2.06.
- kubectl top pods (idle after test): 8phn4 CPU 2m, 376 Mi; h9wr8 CPU 2m, 364 Mi.
- SLO: 95% of /parse requests return 200 within 5 s, over 1 hour. Error budget: 5% of requests. At 3840 req/hour, budget is 192 failed requests per hour. This SLO was NOT met in the Locust run (99.68% failed). The service meets it for single sequential requests (50/50 in 23.7 s locally, and manual /parse via Ingress 200 OK), but not under 4-user non-stop load with the current blocking generate.

## Review table

- Task 1: labelled 3000, kept 2427, buckets created, models uploaded - evidence: results/clean_counts.json, S3 upload log.
- Task 2: v1 loss 2.61 to 0.64, student full frame 32% vs teacher 27% vs base 0% - evidence: training/loss.csv, results/student_v1.json, teacher_validation.json, base.json.
- Task 3: image 1.82 GB, 50/50 local, fault 500 exact - evidence: docker images, /tmp/p*.json, curl output.
- Task 4: HPA 1-2, rollout v1 to v2 to v1, pod delete 11 s, cold start 11 s, load balancing 91/37 - evidence: kubectl get pods/hpa, curl headers.
- Task 5: subnets as above, NACL 6/6 - evidence: tools/nacl_sim.py output.
- Task 6: 6 tests passed, CI file, repo pushed - evidence: pytest output, github.com/aryan-Patel-web/cs3081.
- Task 7: Locust 128/39463 success, cost INR 2.06/1000, SLO not met under load - evidence: results/loadtest_stats.csv, loadtest_headers.json.